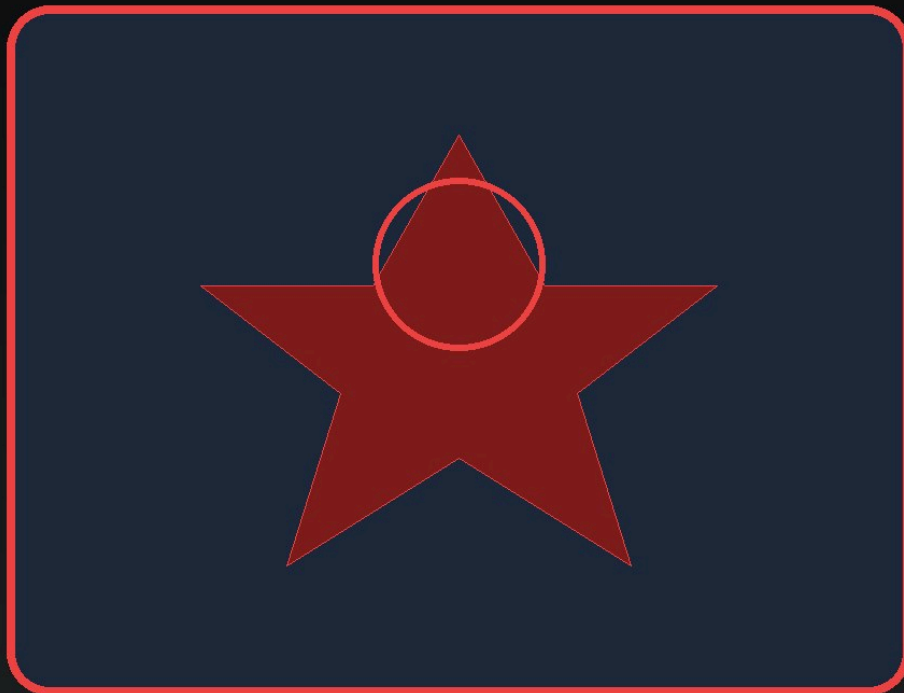




SECURITE WEB - INTERMEDIAIRE

Securite web

Durcir une application web : XSS, CSRF, auth, headers, CSP.

DanielCraft - Defense only - Pack Securite

Sommaire

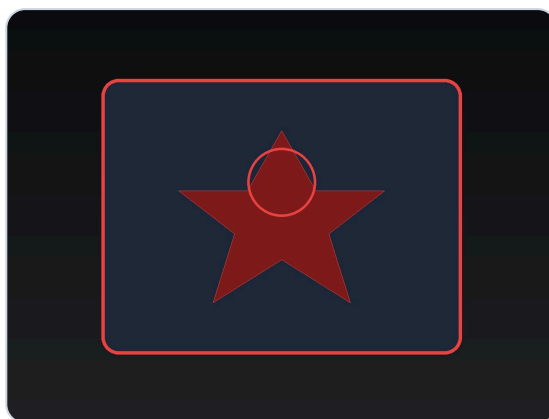
1. Après les bases : aller plus loin
 - Ce que tu vas apprendre
 - A retenir
2. OWASP Top 10 (vue defense)
 - A retenir
3. XSS : comprendre et freiner
 - Types
 - Defense
 - A retenir
4. CSRF : proteger les actions
 - Defense
 - A retenir
5. Injection SQL : requetes preparees
 - Defense
 - Regles
 - A retenir
6. Authentification robuste
 - Principes
 - Flux securise
 - A retenir
7. JWT et sessions
 - JWT (JSON Web Token)
 - Bonnes pratiques
 - Sessions classiques vs JWT
 - A retenir
8. CORS explique
 - Scenario
 - Defense
 - A retenir
9. Content Security Policy (CSP)
 - Effet
 - Demarrage progressif
 - A retenir
10. Rate limiting et brute force
 - Cas d'usage
 - Implementation
 - A retenir
11. Headers de securite
 - A retenir
12. Logging et detection

- [Quoi logger](#)
 - [Quoi NE PAS logger](#)
 - [Alertes](#)
 - [A retenir](#)
13. [Secrets et variables d'environnement](#)
- [Regle d'or](#)
 - [Bonnes pratiques](#)
 - [A retenir](#)
14. [Mini-projet : durcir une petite API](#)
- [Objectif](#)
 - [Checklist](#)
 - [A retenir](#)
15. [Ce qu'il faut retenir](#)
16. [Atelier : tester XSS \(defense\)](#)
- [Exercice](#)
 - [Verification](#)
17. [Atelier : configurer les headers](#)
- [Exercice](#)
 - [Objectif](#)
18. [Erreurs classiques](#)
19. [Bonnes pratiques intermediaire](#)
- [A retenir](#)
20. [Quiz Securite web intermediaire](#)
21. [Bravo !](#)
- [La suite](#)

Après les bases : aller plus loin



Du niveau bases au durcissement.



Couverture Sécurité inter.

Tu connais déjà les habitudes : mots de passe, HTTPS, phishing, mises à jour. Le niveau intermédiaire, c'est **durcir** une application web : comprendre les failles courantes et les **freiner** côté défense.

> **Astuce DanielCraft** - On reste en mode défense. Pas d'exploitation, pas de hacking offensif.

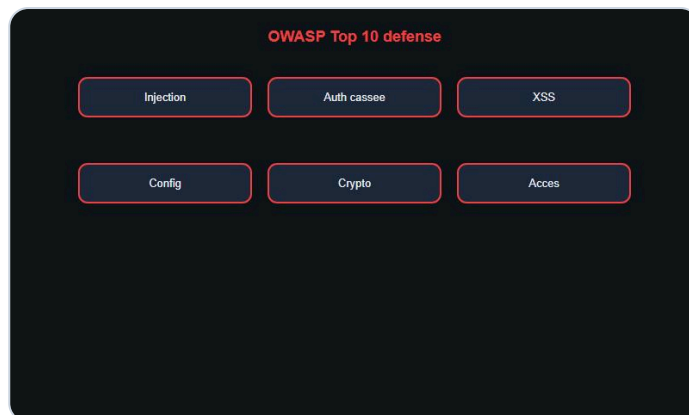
Ce que tu vas apprendre

- OWASP Top 10 (vue défense).
- XSS, CSRF, injections SQL.
- Auth, JWT, sessions.
- Headers, CSP, CORS, rate limiting.

A retenir

- Intermédiaire = appliquer des contre-mesures concrètes.
- Défense only, toujours.

OWASP Top 10 (vue defense)



OWASP Top 10 vue defense.

L'**OWASP Top 10** liste les risques web les plus frequents. Tu n'as pas besoin de tout memoriser : retiens les grandes familles.

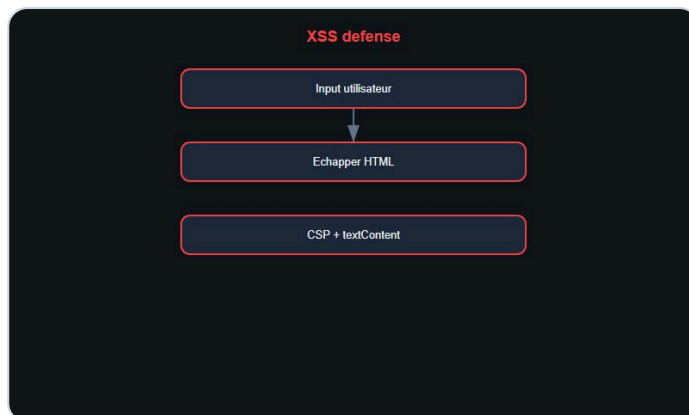
Rang	Risque	Contre-mesure cle
1	Controle d'accès casse	Vérifier les droits à chaque action
2	Faibles cryptographiques	HTTPS, algo modernes, pas de secrets en clair
3	Injection	Requetes preparees, validation
4	Design insecure	Menacer modeler, moindre privilege
5	Mauvaise config	Desactiver debug en prod, headers

> **Astuce DanielCraft** - Utilise la Top 10 comme checklist avant mise en production.

A retenir

- Top 10 = carte des priorites.
- Chaque chapitre suivant couvre une famille.

XSS : comprendre et freiner



Freiner XSS : echapper + CSP.

XSS (Cross-Site Scripting) : un attaquant injecte du JavaScript dans une page vue par d'autres utilisateurs.

Types

- **Reflected** : payload dans l'URL, execute une fois.
- **Stored** : payload stocke en base, execute pour tous.

Defense

1. **Echapper** les sorties HTML (`<` , `>` , etc.).
2. **CSP** (Content Security Policy) pour bloquer scripts non autorises.
3. Ne jamais faire `innerHTML = userInput` sans sanitization.

```
// Mauvais
element.innerHTML = commentaire;

// Mieux
element.textContent = commentaire;
```

A retenir

- XSS = code injecte dans la page.
- Echapper + CSP + textContent.

CSRF : protéger les actions



Token CSRF + SameSite.

CSRF : un site malveillant déclenche une action sur un autre site où tu es déjà connecté (ex. changer ton email).

Defense

1. **Token CSRF** : jeton secret dans le formulaire, vérifie côté serveur.
2. **SameSite cookies** : `SameSite=Strict` ou `Lax`.
3. Vérifier l'**Origin** / **Referer** pour les requêtes sensibles.

```
<input type="hidden" name="csrf_token" value="abc123...">
```

> **Piege** - Les API REST sans token CSRF sont vulnérables si elles utilisent des cookies de session.

A retenir

- CSRF = action forcée depuis un autre site.
- Token + SameSite + vérification Origin.

Injection SQL : requetes preparees



Prepared statements.

Une **injection SQL** arrive quand du texte utilisateur est concatene dans une requete SQL.

```
-- Dangereux
SELECT * FROM users WHERE email = '"' + input + '"'
```

Defense

Requetes preparees (prepared statements) : le SQL et les donnees sont separees.

```
cursor.execute("SELECT * FROM users WHERE email = ?", (email,))
```

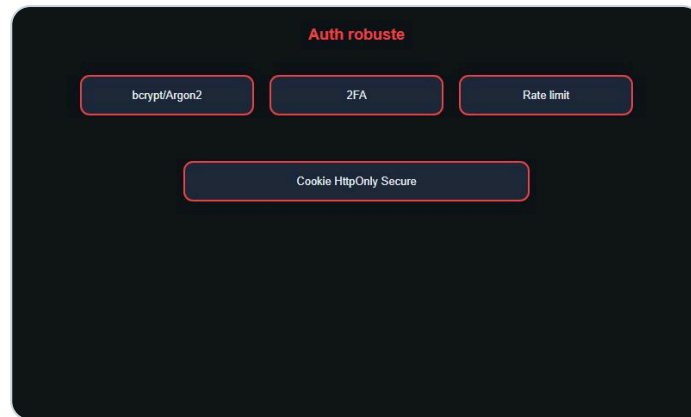
Regles

- Jamais concatener l'input utilisateur dans du SQL.
- Moindre privilege sur le compte DB.
- Valider le format des entrees (email, id numerique).

A retenir

- Prepared statements = regle d'or.
- Valider + moindre privilege DB.

Authentification robuste



Hash + 2FA + rate limit.

Principes

- Mots de passe hashes avec **bcrypt/Argon2** (jamais MD5/SHA1 seuls).
- **2FA** pour les comptes sensibles.
- Limiter les tentatives de connexion (rate limiting).
- Invalider les sessions au logout.

Flux securise

1. Login -> verifier hash + 2FA si active.
2. Creer session avec ID aleatoire long.
3. Cookie **HttpOnly**, **Secure**, **SameSite**.
4. Timeout d'inactivite.

> **Astuce DanielCraft** - Ne reinvente pas l'auth. Utilise un framework ou un service eprouve (OAuth, Auth0, etc.).

A retenir

- Hash fort + 2FA + rate limit + cookies securises.

JWT et sessions



JWT : HttpOnly + expiration.

JWT (JSON Web Token)

Un **JWT** est un token signé contenant des claims (user id, expiration).

```
header.payload.signature
```

Bonnes pratiques

- **Expiration courte** (15 min access + refresh token).
- Stocker en **HttpOnly cookie**, pas localStorage (XSS).
- Verifier la signature a chaque requete.
- Algorithme **RS256** ou **ES256**, pas HS256 avec secret faible.

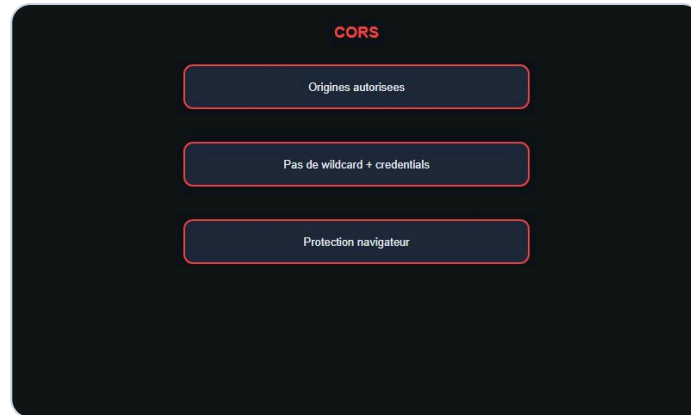
Sessions classiques vs JWT

	Session serveur	JWT
Revocation	Facile	Difficile (blacklist)
Scalabilite	Store partage	Stateless
XSS	Cookie HttpOnly	Attention localStorage

A retenir

- JWT = pratique mais pas magique.
- HttpOnly cookie > localStorage.

CORS explique



Origines autorisées explicites.

CORS (Cross-Origin Resource Sharing) contrôle quels domaines peuvent appeler ton API depuis un navigateur.

Scenario

`https://evil.com` veut appeler `https://api.tonsite.com` avec les cookies de l'utilisateur.

Defense

```
Access-Control-Allow-Origin: https://app.tonsite.com
Access-Control-Allow-Credentials: true
```

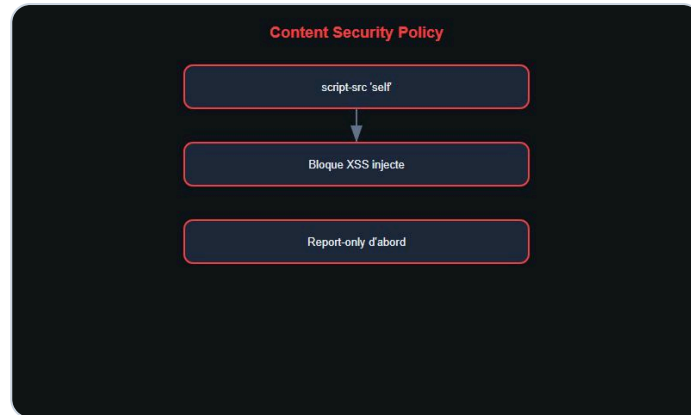
- **Ne jamais** mettre `Access-Control-Allow-Origin: *` avec credentials.
- Lister explicitement les origines autorisées.

> **Piege** - CORS est une protection navigateur, pas serveur. Les appels curl/postman contournent CORS.

A retenir

- CORS = qui peut appeler ton API depuis le browser.
- Origines explicites, pas de wildcard avec cookies.

Content Security Policy (CSP)



Content Security Policy.

La **CSP** dit au navigateur quelles sources de scripts, styles, images sont autorisées.

```
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'
```

Effet

- Bloque les scripts inline injectés (XSS).
- Empêche le chargement de JS depuis des domaines non listés.

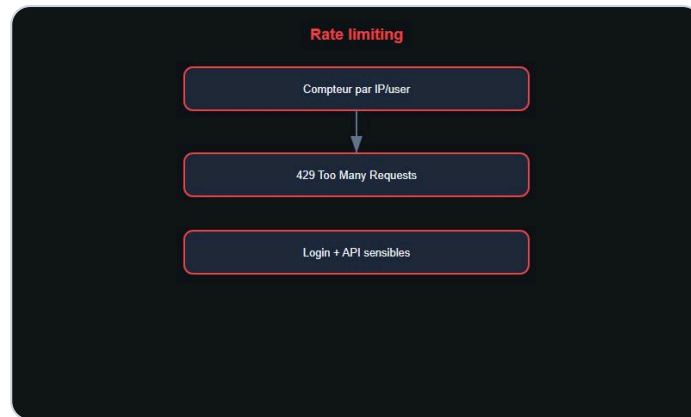
Demarrage progressif

1. Mode **report-only** : `Content-Security-Policy-Report-Only`.
2. Analyser les violations.
3. Passer en mode enforce.

A retenir

- CSP = bouclier anti-XSS puissant.
- Commencer en report-only.

Rate limiting et brute force



429 Too Many Requests.

Le **rate limiting** limite le nombre de requetes par IP/utilisateur sur une fenetre de temps.

Cas d'usage

- Login : 5 tentatives / 15 min.
- API publique : 100 req / min.
- Reset password : 3 / heure.

Implementation

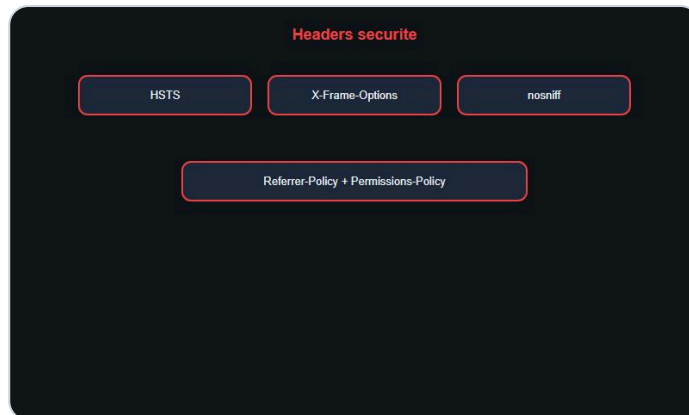
- Compteur en memoire (Redis ideal).
- Reponse 429 Too Many Requests .
- Captcha apres N echecs.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 900
```

A retenir

- Rate limit = frein automatique.
- Protege login, API, actions sensibles.

Headers de securite



HSTS, X-Frame, nosniff.

Headers HTTP qui durcissent le navigateur :

Header	Role
Strict-Transport-Security	Force HTTPS
X-Content-Type-Options: nosniff	Empeche MIME sniffing
X-Frame-Options: DENY	Anti clickjacking
Referrer-Policy	Limite les fuites d'URL
Permissions-Policy	Desactive camera/micro

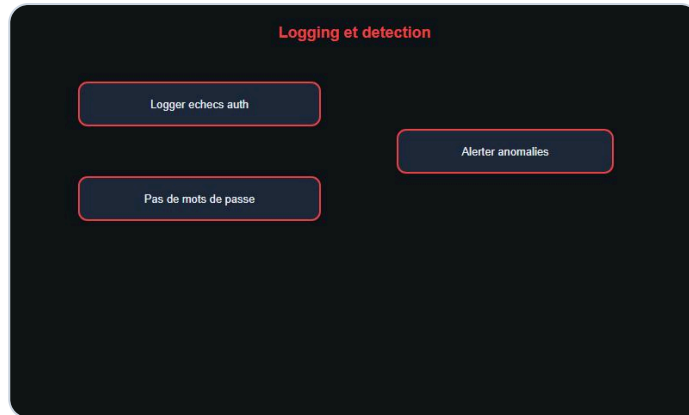
```
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
```

> **Astuce DanielCraft** - Teste tes headers sur securityheaders.com.

A retenir

- Quelques headers bien choisis = gros impact.
- HSTS + X-Frame + nosniff = minimum.

Logging et detection



Logger sans secrets.

Quoi logger

- Tentatives de login echouees.
- Acces admin.
- Erreurs 500.
- Modifications de donnees sensibles.

Quoi NE PAS logger

- Mots de passe.
- Tokens complets.
- Donnees perso inutiles (RGPD).

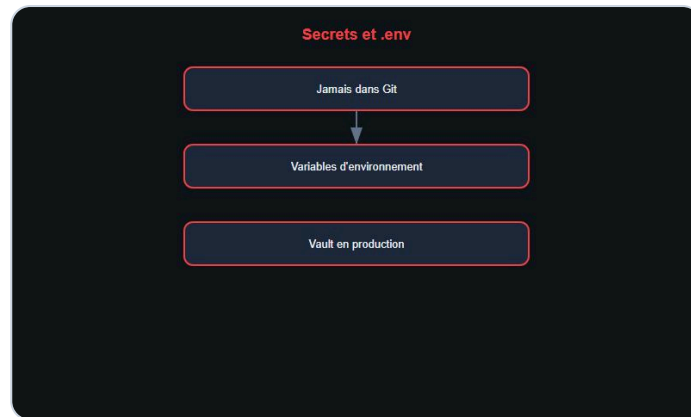
Alertes

- Pic de 401/403.
- Multiples echecs login meme IP.
- Requetes vers des paths suspects (`/admin` , `/.env`).

A retenir

- Logger les evenements de securite, pas les secrets.
- Alerter sur les patterns anormaux.

Secrets et variables d'environnement



Secrets en .env / vault.

Règle d'or

Jamais de secrets dans le code source ou Git.

```
# Mauvais
API_KEY = "sk-abc123..."

# Bon
import os
API_KEY = os.environ["API_KEY"]
```

Bonnes pratiques

- Fichier `.env` local (dans `.gitignore`).
- Vault / secrets manager en production.
- Rotation régulière des clés.
- Séparer dev/staging/prod.

A retenir

- Secrets = variables d'environnement ou vault.
- `.env` jamais commiter.

Mini-projet : durcir une petite API



Durcir une API CRUD.

Objectif

Prendre une API CRUD simple et ajouter :

1. Requetes preparees (anti injection).
2. Rate limiting sur login.
3. Headers de securite (HSTS, X-Frame, nosniff).
4. CSP basique.
5. Validation des entrees.
6. Logging des echecs auth.

Checklist

- Prepared statements partout
- Rate limit login (5/15min)
- Headers securite configures
- CSP en report-only
- Secrets en .env
- Logs sans mots de passe

> **Astuce DanielCraft** - Fais une passe avant/apres. Note chaque amelioration.

A retenir

- Mini-projet = appliquer 6 contre-mesures concretes.

Ce qu'il faut retenir

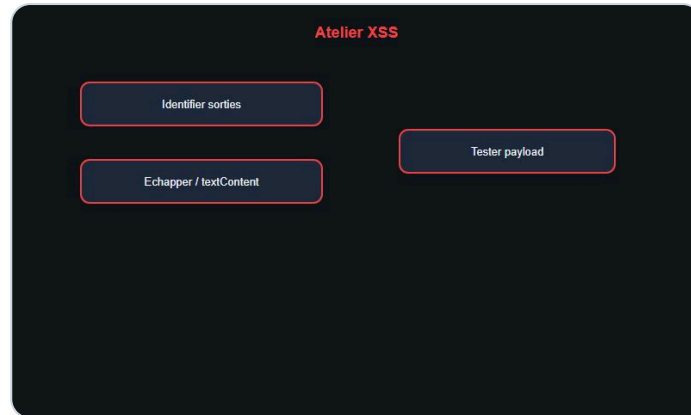


Carte des defenses.

Risque	Defense
XSS	Echapper + CSP
CSRF	Token + SameSite
Injection SQL	Prepared statements
Auth faible	Hash + 2FA + rate limit
JWT mal utilise	HttpOnly + expiration courte
CORS ouvert	Origines explicites
Brute force	Rate limiting
Config	Headers securite
Fuites secrets	.env + vault

> **Astuce DanielCraft** - OWASP Top 10 + checklist headers = base solide avant prod.

Atelier : tester XSS (defense)



Atelier XSS.

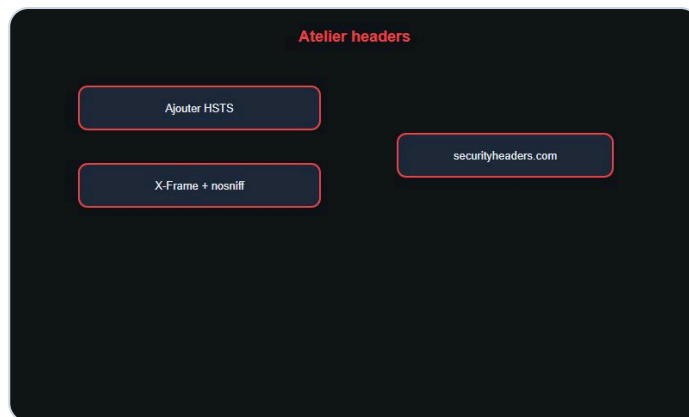
Exercice

1. Identifie 3 endroits où du texte utilisateur est affiché.
2. Pour chacun : échapper ou utiliser `textContent`.
3. Ajoute une CSP : `script-src 'self'`.
4. Teste avec `<script>alert(1)</script>` : doit être neutralisé.

Vérification

- Le script ne s'exécute pas.
- Le texte s'affiche littéralement ou est filtré.

Atelier : configurer les headers



Atelier headers.

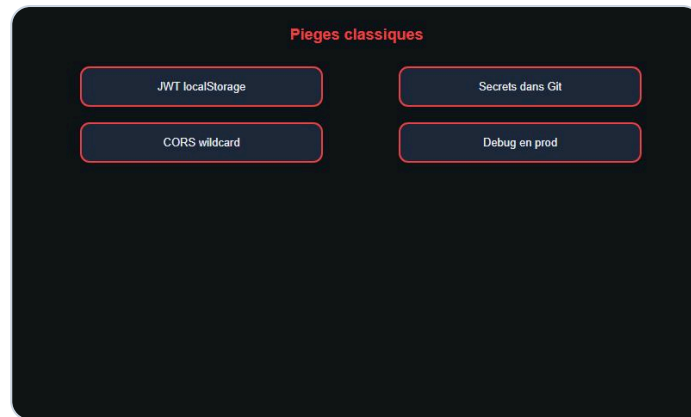
Exercice

1. Ajoute HSTS, X-Frame-Options, X-Content-Type-Options.
2. Verifie avec les DevTools (onglet Network > Response Headers).
3. Teste sur securityheaders.com.
4. Note le score avant/apres.

Objectif

Score A ou B minimum.

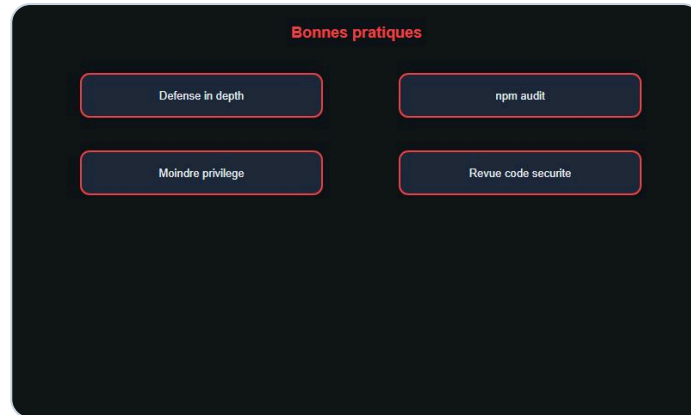
Erreurs classiques



Pieges classiques.

1. **JWT en localStorage** -> vole par XSS.
 2. **CORS wildcard + credentials** -> fuite de session.
 3. **CSP absente** -> XSS non bloque.
 4. **Rate limit absent** -> brute force login.
 5. **Secrets dans Git** -> fuite permanente.
 6. **Debug mode en prod** -> fuite d'infos.
- > **Piege** - La securite n'est pas "faite une fois". Revoie a chaque feature.

Bonnes pratiques intermediaire



Defense in depth.

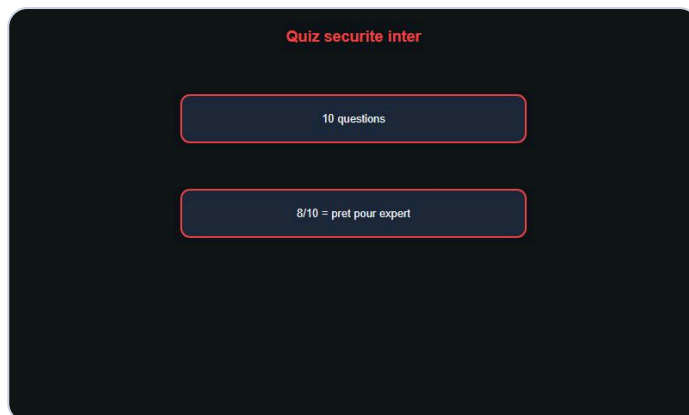
- **Threat modeling** léger : quoi proteger, qui attaque, comment.
- **Defense in depth** : plusieurs couches (validation + prepared + CSP + headers).
- **Principe du moindre privilege** : DB, fichiers, API keys.
- **Revue de code** orientee securite.
- **Dependances** : `npm audit`, Dependabot, mises a jour.

A retenir

- Plusieurs couches > une seule mesure.
- Automatiser les verifications (audit, headers check).

QUIZ

Quiz Securite web intermediaire



Quiz securite inter.

- Q1.** XSS se freine avec : A) MD5 B) Echapper + CSP C) CORS D) JWT → **B**
- Q2.** CSRF token sert a : A) Chiffrer B) Prouver que le formulaire vient du bon site C) Rate limit D) CORS → **B**
- Q3.** Injection SQL : A) Concatener B) Prepared statements C) Base64 D) HTTPS seul → **B**
- Q4.** JWT en localStorage : A) Recommande B) Risque XSS C) Obligatoire D) Meilleur que cookie → **B**
- Q5.** CORS * + credentials : A) OK B) Dangereux C) Obligatoire D) Standard → **B**
- Q6.** HSTS force : A) HTTP B) HTTPS C) CORS D) CSP → **B**
- Q7.** Rate limit retourne souvent : A) 404 B) 429 C) 200 D) 301 → **B**
- Q8.** Secrets en prod : A) Git B) .env commite C) Vault / env vars D) Commentaires → **C**
- Q9.** CSP report-only : A) Bloque B) Observe sans bloquer C) Desactive CSP D) XSS → **B**
- Q10.** OWASP Top 10 : A) Outil hack B) Liste des risques frequents C) Framework D) Certificat → **B**

FINAL

Bravo !



Bravo. Tu durcis sans paniquer.

Tu as termine **Securite web - Intermediaire**. Tu sais freiner XSS, CSRF, injections, durcir l'auth, configurer headers, CSP, CORS et rate limiting.

La suite

- **Securite web - Expert** : pentest legal, architecture zero trust, SIEM.
- > **Astuce DanielCraft** - Applique la checklist OWASP + headers sur un vrai projet cette semaine.
- Tu durcis. Tu proteges. Sans paniquer.

Tu durcis. Tu proteges. Sans paniquer.